

МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего образования
Московский авиационный институт
(Национальный исследовательский университет)

Институт № 8 «Компьютерные науки и прикладная математика»
Кафедра 806

Отчёт по лабораторной работе №3

Дисциплина: «Операционные системы и архитектура компьютера»

Тема: «Работа с потоками»

Вариант: 12

Выполнил:
студент группы М8О-206БВ-25
Ширанков М.С.
Проверил:
Ассистент Центра ТОП-ИТ
Ефимов Д.А.

Москва 2026

Содержание

Цель работы **2**

Задание **2**

Ход работы **2**

Структура программы 2

Ключи запуска 3

Алгоритм 3

Распараллеливание 3

Используемые системные вызовы и функции **5**

Демонстрация работы **5**

Пример фильтрации 5

Количество потоков средствами ОС 6

Трассировка системных вызовов 7

Исследование ускорения и эффективности **8**

Итог работы **10**

Приложение А. Исходный код программы **11**

A.1. main.c 11

A.2. morph.h 17

A.3. morph.c 17

A.4. matrix.h 23

A.5. matrix.c 23

A.6. Makefile 26

Цель работы

Приобрести практические навыки:

- управления потоками в ОС: создания потоков (`pthread_create`), ожидания их завершения (`pthread_join`), ограничения их числа;
- обеспечения синхронизации между потоками: мьютексов, условных переменных, барьеров и атомарных операций.

Задание

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы. Ограничение максимального количества потоков, работающих в один момент времени, должно задаваться ключом запуска программы. Необходимо уметь продемонстрировать количество потоков, используемое программой, стандартными средствами операционной системы. В отчёте привести исследование зависимости ускорения и эффективности алгоритма от входных данных и количества потоков и объяснить полученные результаты.

Вариант 12: наложить K раз фильтры эрозии и наращивания на матрицу, состоящую из вещественных чисел. На выходе получается две результирующие матрицы.

Ход работы

Структура программы

Программа `morph` состоит из трёх модулей (полный исходный код приведён в приложении А):

- `main.c` — разбор ключей запуска, ввод и вывод матриц, замер времени;
- `matrix.c`, `matrix.h` — матрица `matrix_t`: выделение памяти, чтение и запись в текстовом формате, генерация случайной матрицы;
- `morph.c`, `morph.h` — многопоточное применение фильтров: потоки, барьер, распределение работы.

Сборка выполняется по `Makefile` из корня лабораторной работы, замеры для раздела исследования — скриптом `bench/bench.py`:

```
make
./morph -t 4 -k 3 -w 5 -i input.txt -e eroded.txt -d dilated.txt
python3 bench/bench.py
```

Ключи запуска

Формат матрицы: два целых числа — количество строк и столбцов, затем все элементы по строкам через любые пробельные символы (пробелы, табуляции, перевод строки и т.д.). В этом же формате выводятся результаты.

Ключ	Назначение
`-t THREADS`	**обязательный**: максимальное число потоков, работающих одновременно, включая главный (1–`kMaxThreads`)
`-k K`	сколько раз применить каждый фильтр (по умолчанию 1)
`-w WINDOW`	нечётная сторона квадратного окна (по умолчанию 3)
`-i FILE`	файл с матрицей; без ключа матрица читается из `stdin`
`-g RxC`	сгенерировать случайную матрицу $R \times C$ вместо чтения
`-s SEED`	зерно генератора для `-g` (по умолчанию 1)
`-e FILE`, `-d FILE`	файлы для результата эрозии и наращивания; без ключа результат выводится в `stdout`
`-S`	статическое распределение строк между потоками (по умолчанию — динамическое, см. ниже)
`-q`	не выводить результаты, для которых не задан файл
`-v`	вывести в `stderr` PID, число потоков и время фильтрации

Алгоритм

Эрозия и наращивание — операции математической морфологии. Для матрицы A размера $N \times M$ и квадратного окна стороны $w = 2r + 1$:

$$(A \ominus W)_{ij} = \min_{\substack{|a|\le r,\ |b|\le r \\ 0\le i+a<N,\ 0\le j+b<M}} A_{i+a,\,j+b}, \qquad (A \oplus W)_{ij} = \max_{\substack{|a|\le r,\ |b|\le r \\ 0\le i+a<N,\ 0\le j+b<M}} A_{i+a,\,j+b}.$$

У края матрицы окно обрезается (`filter_rows`): учитываются только существующие элементы. Это равносильно дополнению матрицы значением $+\infty$ для эрозии и $-\infty$ для наращивания, поэтому края не искажаются.

Фильтры применяются K раз: $E_0 = D_0 = A$, $E_k = E_{k-1} \ominus W$, $D_k = D_{k-1} \oplus W$; результат — E_K и D_K. Минимум и максимум по окну считаются по определению (`window_min`, `window_max`), поэтому трудоёмкость равна $\Theta(K \cdot N \cdot M \cdot w^2)$ сравнений на каждый фильтр, а размер окна напрямую управляет объёмом вычислений на один элемент.

Распараллеливание

Внутри одной итерации каждый элемент результата зависит только от матрицы предыдущей итерации, поэтому все элементы можно считать независимо. Матрица делится между потоками по строкам; каждую строку результата записывает ровно один поток, и для записи блокировки не нужны. Обе цепочки, эрозия и наращивание, обрабатываются одними и теми же потоками: поток, взявший строки, считает в них оба фильтра.

Двойная буферизация. Для каждого фильтра есть два буфера: итерация i читает буфер итерации $i - 1$ и пишет в `out[i % 2]` (`morph_task_t`, `filter_both`). Первая итерация читает исходную матрицу напрямую, а буферы назначаются так, чтобы последняя, K-я,

запись попала сразу в выходную матрицу (morph_apply). Копирования между итерациями нет.

Барьер. Итерации зависят друг от друга, и опасностей две:

- «чтение до записи»: строки на границе участка читают строки соседнего участка, и итерация $i+1$ не может начаться, пока сосед не дописал их на итерации i;
- «запись до окончания чтения»: итерация $i+1$ пишет в буфер `out[(i+1) % 2]`, который на итерации i ещё читают другие потоки как результат итерации $i-1$.

Обе снимает один барьер между итерациями: никакой поток не начинает итерацию $i+1$, пока все не закончили итерацию i. После последней итерации барьер не нужен — его роль играет `pthread_join`.

В macOS нет `pthread_barrier_t`, поэтому барьер barrier_t реализован на мьютексе и условной переменной (barrier_wait). Поток увеличивает счётчик пришедших; последний обнуляет его, увеличивает номер поколения (`generation`) и будит остальных через `pthread_cond_broadcast`. Остальные ждут в цикле, пока не сменится поколение. Номер поколения нужен по двум причинам: `pthread_cond_wait` допускает ложные пробуждения, а поток, быстро прошедший барьер, может прийти к нему снова раньше, чем проснулись остальные, — проверка счётчика вместо поколения в этом случае сломалась бы.

Ограничение числа потоков. Ключ `-t` задаёт число T; создаётся $\min(T, N)$ вычисляющих потоков, так как делить работу мельче строки нет смысла. Главный поток не простаивает в `pthread_join`, а работает как поток номер 0 (run_workers), поэтому в процессе ровно $\min(T, N)$ потоков. Потоки создаются один раз и живут все K итераций; создавать их заново на каждой итерации было бы дороже.

Перед вычислениями все потоки ждут на стартовом барьере (worker_main). Если `pthread_create` не удался, уже запущенные потоки ждали бы на барьере вечно: число участников не набралось бы. Поэтому барьер можно «сломать» (barrier_break): ожидающие просыпаются, получают -1 и завершаются, а главный поток дожидается их и сообщает об ошибке.

Распределение строк. Реализованы две стратегии:

- **статическая** (ключ `-S`): поток t из T получает полосу строк $[\lfloor tN/T \rfloor, \lfloor (t+1)N/T \rfloor)$; полосы отличаются по высоте не больше чем на одну строку;
- **динамическая** (по умолчанию): матрица делится на блоки по $\lceil N/(16T) \rceil$ строк (kChunksPerThread), а поток, закончив блок, берёт номер следующего атомарной операцией `atomic_fetch_add` над общим счётчиком. Быстрый поток успевает обработать больше блоков, чем медленный.

Счётчик блоков после итерации нужно обнулить, но сделать это сразу нельзя: другие потоки ещё могут обращаться к нему. Поэтому счётчиков два, как и буферов: итерация i раздаёт блоки из `next_chunk[i % 2]`, а поток 0 в её начале обнуляет `next_chunk[(i+1) % 2]` (run_iteration). Этот счётчик использовался на итерации $i-1$, и все потоки уже прошли барьер после неё; обнуление станет видно остальным через барьер после итерации i. Порядок обращений к счётчику обеспечивает барьер на мьютексе, поэтому для самой атомарной операции достаточно `memory_order_relaxed`.

Используемые системные вызовы и функции

Вызов	Назначение в программе	Где в коде
`pthread_create`	создание вычисляющих потоков	`run_workers`
`pthread_join`	ожидание завершения потоков и освобождение их ресурсов	`run_workers`
`pthread_mutex_init`, `_lock`, `_unlock`, `_destroy`	защита состояния барьера	`barrier_t`, `barrier_wait`
`pthread_cond_init`, `_wait`, `_broadcast`, `_destroy`	ожидание остальных потоков на барьере и их пробуждение	`barrier_wait`, `barrier_break`
`atomic_fetch_add`, `atomic_store`	выдача блоков строк при динамическом распределении	`run_iteration`
`clock_gettime`	замер времени фильтрации по монотонным часам (`CLOCK_MONOTONIC`)	`filter`
`malloc`, `free`	память под матрицы и промежуточные буферы	`matrix_alloc`, `morph_apply`
`fopen`, `fscanf`, `fprintf`, `fclose`	чтение исходной и запись результирующих матриц	`load_matrix`, `matrix_read`, `matrix_write`
`fstat`	проверка, что результаты не записываются в один файл	`is_same_file`
`getopt`	разбор ключей запуска	`parse_options`

На уровне ядра `pthread_create` в Linux выполняется системным вызовом `clone3` с флагами `CLONE_VM`, `CLONE_THREAD` и др.: новый поток разделяет с создателем адресное пространство, дескрипторы и обработчики сигналов. Ожидание на мьютексе и условной переменной при конфликте превращается в вызов `futex`; без конфликта мьютекс захватывается атомарной операцией в пространстве пользователя, без системного вызова.

Демонстрация работы

Пример фильтрации

Матрица 4×5, окно 3×3. При $K = 1$ оба результата выводятся в `stdout` (сначала эрозия, затем наращивание), при $K = 2$ — в файлы:

```
$ cat input.txt
4 5
1 9 2 8 3
7 0.5 6 4 5
2 3 -1 9 6
8 4 7 1 2
$ ./morph -t 2 -w 3 -k 1 -i input.txt
4 5
0.5 0.5 0.5 2 3
0.5 -1 -1 -1 3
0.5 -1 -1 -1 1
2 -1 -1 -1 1
```

```
4 5
9 9 9 8 8
9 9 9 9 9
8 8 9 9 9
8 8 9 9 9
$ ./morph -t 2 -w 3 -k 2 -i input.txt -e eroded.txt -d dilated.txt -v
pid 71752: filtering 4x5...
matrix=4x5 window=3 iterations=2 threads=2 split=dynamic time_ms=0.084
$ cat eroded.txt dilated.txt
4 5
-1 -1 -1 -1 -1
-1 -1 -1 -1 -1
-1 -1 -1 -1 -1
-1 -1 -1 -1 -1
4 5
9 9 9 9 9
9 9 9 9 9
9 9 9 9 9
8 9 9 9 9
```

Например, элемент $(0, 0)$ после эрозии равен $\min(1, 9, 7, 0{,}5) = 0{,}5$: окно в углу обрезано до 2×2. Эрозия «растягивает» минимумы, а наращивание — максимумы: каждый проход сдвигает их границу на $r = 1$ элемент. От минимума -1 в позиции $(2, 2)$ до любого элемента матрицы не больше двух шагов по строкам и столбцам, поэтому за два прохода он заполнил всю матрицу. А до элемента $(3, 0)$ от обеих девяток, $(0, 1)$ и $(2, 3)$, три шага, поэтому после наращивания там осталось 8.

Количество потоков средствами ОС

Программа разрабатывалась на macOS (Apple M1). Запускаем долгую фильтрацию с ограничением в 6 потоков и смотрим на процесс из другого терминала. Утилита `ps` с ключом `-M` выводит по строке на каждый поток, а `top` — число потоков в столбце `#TH`:

```
$ ./morph -t 6 -g 3000x3000 -w 9 -k 30 -q -v &
pid 74665: filtering 3000x3000...
$ ps -M -p 74665
USER       PID   TT   %CPU STAT PRI     STIME     UTIME COMMAND
eblackbu 74665   ??   66.3 R    31T   0:00.04   0:00.79 ./morph -t 6 -g 3000x3000 -w 9
     -k 30 -q -v
         74665        70.0 R    31T   0:00.04   0:00.80
         74665        67.9 R    31T   0:00.03   0:00.77
         74665        66.8 R    31T   0:00.03   0:00.77
         74665        65.7 R    31T   0:00.03   0:00.75
         74665        67.5 R    31T   0:00.03   0:00.78
$ top -l 1 -pid 74665 -stats pid,command,cpu,threads
PID    COMMAND %CPU #TH
74665  morph   0.0  6/6
```

Потоков ровно шесть: главный и пять созданных, все в состоянии `R` (выполняется) с почти одинаковым процессорным временем `UTIME` — работа распределена равномерно. (`top -l 1` делает один снимок, поэтому загрузка в нём не измеряется и показана как 0.)

То же в Linux (контейнер Debian, ядро 6.12, `gcc` 14.2). Здесь каждый поток — отдельная задача ядра со своим идентификатором `SPID` (`TID`):

```
$ ps -T -p 611
```

```
  PID  SPID TTY          TIME CMD
  611   611 ?        00:00:01 morph
  611   613 ?        00:00:01 morph
  611   614 ?        00:00:01 morph
  611   615 ?        00:00:01 morph
  611   616 ?        00:00:01 morph
  611   617 ?        00:00:01 morph
$ ps -o pid,nlwp,cmd -p 611
  PID NLWP CMD
  611    6 ./morph -t 6 -g 3000x3000 -w 9 -k 30 -q -v
$ grep Threads /proc/611/status
Threads:  6
$ ls /proc/611/task
611  613  614  615  616  617
```

Также число потоков видно в `top -H` и `htop`. Если строк в матрице меньше, чем задано `-t`, потоков создаётся по числу строк.

Трассировка системных вызовов

```
strace -f -e trace=clone3,futex,exit,exit_group ./morph -t 3 -g 600x600 -k 2 -q
```

Трасса (PID 625 — главный поток, 626 и 627 — созданные; длинные аргументы сокращены):

```
625   clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|
              CLONE_SYSVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
              ..., stack_size=0x80ea60, ...} => {parent_tid=[626]}, 88) = 626
626   futex(0xffffc4d83010, FUTEX_WAIT_BITSET_PRIVATE|..., 0, ...) <unfinished ...>
625   clone3({flags=...|CLONE_THREAD|..., ...} => {parent_tid=[627]}, 88) = 627
625   futex(0xffffc4d83010, FUTEX_WAIT_BITSET_PRIVATE|..., 0, ...) <unfinished ...>
627   futex(0xffffc4d83010, FUTEX_WAKE_PRIVATE, 2147483647) = 2
625   <... futex resumed>)              = 0
626   <... futex resumed>)              = 0
625   futex(0xffffc4d82fc0, FUTEX_WAKE_PRIVATE, 1 <unfinished ...>
626   futex(0xffffc4d82fc0, FUTEX_WAKE_PRIVATE, 1 <unfinished ...>
...
627   futex(0xffffc4d83014, FUTEX_WAIT_BITSET_PRIVATE|..., 0, ...) <unfinished ...>
626   futex(0xffffc4d83014, FUTEX_WAIT_BITSET_PRIVATE|..., 0, ...) <unfinished ...>
625   futex(0xffffc4d83014, FUTEX_WAKE_PRIVATE, 2147483647) = 2
...
625   futex(0xffff84b9f250, FUTEX_WAIT_BITSET|..., 626, ...) <unfinished ...>
627   exit(0)                           = ?
626   exit(0 <unfinished ...>
625   <... futex resumed>)              = 0
625   exit_group(0)                     = ?
```

По трассе видно:

- `pthread_create` — это `clone3` с флагами `CLONE_VM` (общая память), `CLONE_FILES` (общие дескрипторы), `CLONE_SIGHAND` и `CLONE_THREAD` (тот же процесс), под стек нового потока выделено около 8 МБ (`stack_size=0x80ea60`);
- барьер проходится дважды: стартовый и между двумя итерациями. На первом последним пришёл поток 627, на втором — главный поток 625. Пришедшие раньше спят в `FUTEX_WAIT` на условной переменной (адреса `...3010` и `...3014` — её

внутренний счётчик), последний будит всех одним FUTEX_WAKE с числом $2^{31} - 1$ (pthread_cond_broadcast), ядро сообщает, что проснулись два потока;

- проснувшиеся потоки одновременно захватывают мьютекс барьера (...2fc0), отсюда короткие FUTEX_WAKE на нём. Без конфликта мьютекс системных вызовов не делает — в трассе нет захватов при чтении счётчика блоков, он атомарный;
- pthread_join ждёт поток на futex по адресу его идентификатора: ядро обнуляет его и будит ожидающего при выходе потока (флаг CLONE_CHILD_CLEARTID);
- созданные потоки завершаются вызовом exit, который завершает только поток, а главный — exit_group, завершающим весь процесс.

Сводная статистика strace -f -c для 1000×1000, $K = 10$, 4 потока: 93 вызова futex (99% системного времени, это время ожидания на барьере) и 3 вызова clone3. Вызовов на элемент матрицы нет совсем: вся фильтрация выполняется в пространстве пользователя, ядро участвует только в создании потоков и в их ожидании.

Исследование ускорения и эффективности

Ускорение и эффективность на p потоках:

$$S_p = \frac{T_1}{T_p}, \qquad E_p = \frac{S_p}{p},$$

где T_p — время фильтрации на p потоках (включая создание и завершение потоков), T_1 — время на одном потоке.

Замеры выполнены на MacBook Air (Apple M1: 4 производительных ядра и 4 энергоэффективных, примерно в 2–3 раза более медленных). Скрипт bench/bench.py запускает каждую конфигурацию 9 раз в случайном порядке и берёт медиану времени; результаты сохраняются в bench/results/*.csv. Сводная таблица по этим результатам:

Таблица 1: Время на одном потоке, ускорение и эффективность

Параметры	**Распр.**	T_1**, мс**	S_2	S_4	S_8	S_{16}	E_4	E_8
Размер матрицы $N \times N$*, окно* 3×3*,* $K = 10$								
$N = 50$	стат.	0,52	1,34	1,66	0,94	0,57	0,41	0,12
	дин.	0,52	1,49	1,59	0,78	0,49	0,40	0,10
$N = 100$	стат.	2,09	1,77	2,57	2,18	1,91	0,64	0,27
	дин.	2,09	1,78	2,91	2,19	1,65	0,73	0,27
$N = 200$	стат.	8,35	1,93	3,46	4,05	3,50	0,86	0,51
	дин.	8,35	1,94	3,55	4,14	3,33	0,89	0,52
$N = 500$	стат.	52,1	1,98	3,86	4,79	4,90	0,96	0,60
	дин.	52,2	1,94	3,84	5,62	5,43	0,96	0,70
$N = 1000$	стат.	210	1,99	3,86	5,26	5,11	0,96	0,66
	дин.	210	1,96	3,85	5,74	5,91	0,96	0,72
$N = 2000$	стат.	846	1,99	3,81	4,64	4,83	0,95	0,58
	дин.	845	1,95	3,76	5,40	5,90	0,94	0,68
Размер окна w*, матрица* 1000×1000*,* $K = 4$								
$w = 1$	стат.	20,2	1,85	3,28	2,81	2,81	0,82	0,35
	дин.	20,2	1,86	3,29	2,87	2,57	0,82	0,36
$w = 3$	стат.	87,1	2,00	3,82	5,00	5,01	0,96	0,63
	дин.	86,6	1,96	3,70	5,45	5,30	0,93	0,68
$w = 5$	стат.	190	2,00	3,93	5,22	5,25	0,98	0,65
	дин.	193	1,97	3,73	6,02	6,06	0,93	0,75
$w = 9$	стат.	816	2,04	4,04	6,01	6,04	1,01	0,75
	дин.	816	2,00	3,99	6,33	6,67	1,00	0,79
$w = 15$	стат.	3150	2,00	3,98	6,94	6,82	1,00	0,87
	дин.	3153	1,97	3,91	7,29	7,50	0,98	0,91

Выводы:

- **До 4 потоков** ускорение почти линейное ($E_4 \approx 0{,}94$–$1{,}0$ для $N \geq 500$ и $w \geq 3$): потоки пишут в разные строки, а синхронизация — один барьер на итерацию.
- **Малые матрицы** ускоряются плохо: создание потока стоит около 14 мкс, проход барьера — десятки микросекунд, а итерация для 50×50 длится всего около 52 мкс. Поэтому уже на 8 потоках программа медленнее последовательной ($S_8 < 1$).
- **От 5 до 8 потоков** подключаются медленные E-ядра. При статическом распределении время итерации определяет поток на E-ядре, а остальные ждут его на барьере; динамическое распределение отдаёт быстрым потокам больше блоков и выигрывает (для $N = 1000$: $S_8 = 5{,}74$ против 5,26).
- **Больше 8 потоков** — больше, чем ядер: ускорение не растёт, а эффективность падает.
- **Чем больше окно**, тем больше вычислений приходится на одну синхронизацию и тем ближе ускорение к идеальному: при $w = 15$ получено $S_8 = 7{,}29$. Окно 1×1 — это просто копирование, оно упирается в память, и ускорение не превышает 3,3.

Итог работы

В ходе лабораторной работы реализована программа многократного применения фильтров эрозии и наращивания к матрице вещественных чисел с использованием потоков POSIX. Число одновременно работающих потоков задаётся ключом `-t`, а их фактическое количество продемонстрировано средствами macOS (`ps -M`, `top`) и Linux (`ps -T`, `/proc`). Получены практические навыки создания и ожидания потоков (`pthread_create`, `pthread_join`) и их синхронизации: собственный барьер на мьютексе и условной переменной, атомарный счётчик для распределения работы.

ПРИЛОЖЕНИЕ А

Исходный код программы

Исходный код также доступен в репозитории на GitHub.

A.1. main.c

```c
#include <errno.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

#include "matrix.h"
#include "morph.h"

// Коды завершения программы.
enum {
    kExitUsage = 1, // неверные ключи запуска
    kExitInput = 2, // ошибка чтения или формата входной матрицы
    kExitCompute = 3, // нехватка памяти, ошибка создания потоков
    kExitOutput = 4, // ошибка открытия или записи выходного файла
};

static const size_t kMaxThreads = 4096;

typedef struct {
    morph_params_t morph;
    const char* input_path;
    const char* eroded_path;
    const char* dilated_path;
    size_t gen_rows; // 0, если матрица читается, а не генерируется
    size_t gen_cols;
    uint64_t seed;
    int quiet;
    int verbose;
} options_t;

static void print_usage(const char* prog) {
    fprintf(
        stderr,
        "Usage: %s -t THREADS [-k K] [-w WINDOW] [-i FILE | -g ROWSxCOLS [-s SEED]]\n"
        " [-e FILE] [-d FILE] [-S] [-q] [-v]\n"
        "Applies erosion and dilation K times to a matrix of real numbers.\n"
        " -t THREADS max number of threads working at the same time (1..%zu)\n"
        " -k K number of filter passes (default 1)\n"
        " -w WINDOW odd side of the square window (default 3)\n"
        " -i FILE read the matrix from FILE (default stdin): \"rows cols\" then values\
          n"
        " -g RxC generate a random R x C matrix instead of reading one\n"
        " -s SEED seed for -g (default 1)\n"
        " -e FILE write the eroded matrix to FILE (default stdout)\n"
        " -d FILE write the dilated matrix to FILE (default stdout)\n"
```

```
        " -S static split: one band of rows per thread (default: rows are\n"
        " handed out to threads in small chunks as they finish)\n"
        " -q do not print results that have no output file\n"
        " -v print pid, used threads and filtering time to stderr\n",
        prog,
        kMaxThreads
    );
}

static int parse_size(const char* text, size_t* value, char** rest) {
    if (text[0] < '0' || text[0] > '9') {
        return -1;
    }
    char* end = NULL;
    errno = 0;
    unsigned long long parsed = strtoull(text, &end, 10);
    if (errno == ERANGE || parsed > SIZE_MAX || (rest == NULL && *end != '\0')) {
        return -1;
    }
    if (rest != NULL) {
        *rest = end;
    }
    *value = (size_t)parsed;
    return 0;
}

static int parse_option_size(char opt, const char* text, size_t min, size_t max,
    size_t* value) {
    if (parse_size(text, value, NULL) != 0 || *value < min || *value > max) {
        fprintf(stderr, "Option -%c: expected an integer in %zu..%zu, got '%s'.\n", opt
            , min, max, text);
        return -1;
    }
    return 0;
}

static int parse_shape(const char* text, size_t* rows, size_t* cols) {
    char* rest = NULL;
    if (parse_size(text, rows, &rest) != 0 || (*rest != 'x' && *rest != 'X') ||
        parse_size(rest + 1, cols, NULL) != 0) {
        fprintf(stderr, "Option -g: expected ROWSxCOLS, got '%s'.\n", text);
        return -1;
    }
    return 0;
}

static int parse_options(int argc, char* argv[], options_t* options) {
    *options = (options_t){
        .morph = {.iterations = 1, .window = 3, .max_threads = 0},
        .seed = 1,
    };

    int opt = 0;
    size_t seed = 0;
    while ((opt = getopt(argc, argv, "t:k:w:i:g:s:e:d:Sqvh")) != -1) {
        int status = 0;
        switch (opt) {
            case 't':
```

```
104                 status = parse_option_size('t', optarg, 1, kMaxThreads, &options->morph
                        .max_threads);
105                 break;
106             case 'k':
107                 status = parse_option_size('k', optarg, 0, SIZE_MAX, &options->morph.
                        iterations);
108                 break;
109             case 'w':
110                 status = parse_option_size('w', optarg, 1, SIZE_MAX, &options->morph.
                        window);
111                 if (status == 0 && options->morph.window % 2 == 0) {
112                     fprintf(stderr, "Option -w: window side must be odd to have a center
                            , got %s.\n", optarg);
113                     status = -1;
114                 }
115                 break;
116             case 'i':
117                 options->input_path = optarg;
118                 break;
119             case 'g':
120                 status = parse_shape(optarg, &options->gen_rows, &options->gen_cols);
121                 break;
122             case 's':
123                 status = parse_option_size('s', optarg, 0, SIZE_MAX, &seed);
124                 options->seed = seed;
125                 break;
126             case 'e':
127                 options->eroded_path = optarg;
128                 break;
129             case 'd':
130                 options->dilated_path = optarg;
131                 break;
132             case 'S':
133                 options->morph.static_split = 1;
134                 break;
135             case 'q':
136                 options->quiet = 1;
137                 break;
138             case 'v':
139                 options->verbose = 1;
140                 break;
141             case 'h':
142                 return 1;
143             default:
144                 status = -1;
145         }
146         if (status != 0) {
147             return -1;
148         }
149     }
150
151     if (optind < argc) {
152         fprintf(stderr, "Unexpected argument '%s'.\n", argv[optind]);
153         return -1;
154     }
155     if (options->morph.max_threads == 0) {
156         fprintf(stderr, "The maximum number of threads must be set with -t.\n");
157         return -1;
```

```
158     }
159     if (options->input_path != NULL && options->gen_rows != 0) {
160         fprintf(stderr, "Options -i and -g cannot be used together.\n");
161         return -1;
162     }
163     return 0;
164 }
165 
166 static int load_matrix(const options_t* options, matrix_t* matrix) {
167     if (options->gen_rows != 0) {
168         if (matrix_alloc(matrix, options->gen_rows, options->gen_cols) != 0) {
169             return -1;
170         }
171         matrix_fill_random(matrix, options->seed);
172         return 0;
173     }
174 
175     if (options->input_path == NULL) {
176         if (isatty(STDIN_FILENO)) {
177             fprintf(stderr, "Enter the matrix: rows cols, then the values (Ctrl+D to
                finish):\n");
178         }
179         return matrix_read(stdin, matrix);
180     }
181 
182     FILE* input = fopen(options->input_path, "r");
183     if (input == NULL) {
184         fprintf(stderr, "Cannot open '%s': %s\n", options->input_path, strerror(errno))
                ;
185         return -1;
186     }
187     int result = matrix_read(input, matrix);
188     fclose(input);
189     return result;
190 }
191 
192 // без имени файла результат идёт в stdout, а с -q не выводится (*out == NULL)
193 static int open_output(const char* path, int quiet, FILE** out) {
194     *out = NULL;
195     if (path == NULL) {
196         if (!quiet) {
197             *out = stdout;
198         }
199         return 0;
200     }
201     *out = fopen(path, "w");
202     if (*out == NULL) {
203         fprintf(stderr, "Cannot open '%s' for writing: %s\n", path, strerror(errno));
204         return -1;
205     }
206     return 0;
207 }
208 
209 static int close_output(FILE* out, const char* path) {
210     if (out == NULL || out == stdout) {
211         return 0;
212     }
213     if (fclose(out) != 0) {
```

```
214         fprintf(stderr, "Cannot write '%s': %s\n", path, strerror(errno));
215         return -1;
216     }
217     return 0;
218 }
219 
220 static int is_same_file(FILE* first, FILE* second) {
221     if (first == NULL || second == NULL || first == stdout || second == stdout) {
222         return 0;
223     }
224     struct stat first_stat;
225     struct stat second_stat;
226     if (fstat(fileno(first), &first_stat) != 0 || fstat(fileno(second), &second_stat)
            != 0) {
227         perror("fstat");
228         return 1;
229     }
230     if (first_stat.st_dev == second_stat.st_dev && first_stat.st_ino == second_stat.
            st_ino &&
231         S_ISREG(first_stat.st_mode))
232     {
233         fprintf(stderr, "Erosion and dilation cannot be written to the same file.\n");
234         return 1;
235     }
236     return 0;
237 }
238 
239 static int write_output(FILE* out, const char* path, const matrix_t* matrix) {
240     if (out != NULL && matrix_write(out, matrix) != 0) {
241         fprintf(stderr, "Cannot write '%s': %s\n", path != NULL ? path : "stdout",
                strerror(errno));
242         return -1;
243     }
244     return 0;
245 }
246 
247 static double elapsed_ms(const struct timespec* start, const struct timespec* end) {
248     return ((double)(end->tv_sec - start->tv_sec) * 1e3) + ((double)(end->tv_nsec -
            start->tv_nsec) / 1e6);
249 }
250 
251 static int filter(const options_t* options, const matrix_t* src, matrix_t* eroded,
        matrix_t* dilated) {
252     if (matrix_alloc(eroded, src->rows, src->cols) != 0 || matrix_alloc(dilated, src->
            rows, src->cols) != 0) {
253         return -1;
254     }
255     if (options->verbose) {
256         fprintf(stderr, "pid %d: filtering %zux%zu...\n", (int)getpid(), src->rows, src
                ->cols);
257     }
258 
259     struct timespec start;
260     struct timespec end;
261     size_t threads_used = 0;
262     clock_gettime(CLOCK_MONOTONIC, &start);
263     int result = morph_apply(src, eroded, dilated, &options->morph, &threads_used);
264     clock_gettime(CLOCK_MONOTONIC, &end);
```

```
265
266     if (result == 0 && options->verbose) {
267         fprintf(
268             stderr,
269             "matrix=%zux%zu window=%zu iterations=%zu threads=%zu split=%s time_ms=%.3f
                \n",
270             src->rows,
271             src->cols,
272             options->morph.window,
273             options->morph.iterations,
274             threads_used,
275             options->morph.static_split ? "static" : "dynamic",
276             elapsed_ms(&start, &end)
277         );
278     }
279     return result;
280 }
281
282 static int run(const options_t* options) {
283     FILE* eroded_out = NULL;
284     FILE* dilated_out = NULL;
285     if (open_output(options->eroded_path, options->quiet, &eroded_out) != 0 ||
286         open_output(options->dilated_path, options->quiet, &dilated_out) != 0 ||
                is_same_file(eroded_out, dilated_out))
287     {
288         close_output(eroded_out, options->eroded_path);
289         close_output(dilated_out, options->dilated_path);
290         return kExitOutput;
291     }
292
293     matrix_t src = {0};
294     matrix_t eroded = {0};
295     matrix_t dilated = {0};
296     int code = 0;
297     if (load_matrix(options, &src) != 0) {
298         code = kExitInput;
299     } else if (filter(options, &src, &eroded, &dilated) != 0) {
300         code = kExitCompute;
301     } else if (
302         write_output(eroded_out, options->eroded_path, &eroded) != 0 ||
303         write_output(dilated_out, options->dilated_path, &dilated) != 0
304     )
305     {
306         code = kExitOutput;
307     }
308
309     if (close_output(eroded_out, options->eroded_path) != 0) {
310         code = kExitOutput;
311     }
312     if (close_output(dilated_out, options->dilated_path) != 0) {
313         code = kExitOutput;
314     }
315     matrix_free(&src);
316     matrix_free(&eroded);
317     matrix_free(&dilated);
318     return code;
319 }
320
```

```c
int main(int argc, char* argv[]) {
    const char* prog = argc > 0 ? argv[0] : "morph";
    options_t options;
    int parsed = parse_options(argc, argv, &options);
    if (parsed != 0) {
        print_usage(prog);
        return parsed > 0 ? 0 : kExitUsage;
    }
    return run(&options);
}
```

A.2. morph.h

```c
#ifndef MORPH_H
#define MORPH_H

#include <stddef.h>

#include "matrix.h"

typedef struct {
    size_t iterations;
    size_t window; // сторона квадратного окна, нечётная
    size_t max_threads; // максимум одновременно работающих потоков, включая главный
    int static_split; // 1: каждому потоку одна полоса строк, 0: блоки строк раздаются
        по мере готовности
} morph_params_t;

// В *threads_used записывается число потоков, реально выполнявших вычисления
int morph_apply(
    const matrix_t* src,
    matrix_t* eroded,
    matrix_t* dilated,
    const morph_params_t* params,
    size_t* threads_used
);

#endif
```

A.3. morph.c

```c
#include "morph.h"

#include <pthread.h>
#include <stdatomic.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "matrix.h"

typedef struct {
    pthread_mutex_t mutex;
    pthread_cond_t cond;
    size_t count;
    size_t waiting;
    unsigned long generation;
```

```
17      int broken;
18  } barrier_t;
19
20  typedef enum {
21      kErode,
22      kDilate,
23  } morph_op_t;
24
25  // Полуинтервал строк [begin, end)
26  typedef struct {
27      size_t begin;
28      size_t end;
29  } row_range_t;
30
31  // границы включительно
32  typedef struct {
33      size_t top;
34      size_t bottom;
35      size_t left;
36      size_t right;
37  } window_t;
38
39  // сколько блоков строк приходится в среднем на поток при динамическом распределении
40  static const size_t kChunksPerThread = 16;
41
42  // Общие для всех потоков данные. Буферы результата каждого фильтра чередуются:
43  // итерация i пишет в out[i % 2] и читает то, что записала итерация i - 1.
44  // Так же чередуются счётчики блоков: next_chunk[i % 2] выдаёт блоки итерации i.
45  typedef struct {
46      size_t rows;
47      size_t cols;
48      size_t radius;
49      size_t iterations;
50      int dynamic;
51      size_t chunk_rows;
52      size_t chunks;
53      atomic_size_t next_chunk[2];
54      const double* src;
55      double* eroded[2];
56      double* dilated[2];
57      barrier_t barrier;
58  } morph_task_t;
59
60  // При статическом распределении поток обрабатывает полосу строк band.
61  typedef struct {
62      morph_task_t* task;
63      size_t index;
64      row_range_t band;
65      pthread_t thread;
66  } worker_t;
67
68  static int barrier_init(barrier_t* barrier, size_t count) {
69      int err = pthread_mutex_init(&barrier->mutex, NULL);
70      if (err != 0) {
71          return err;
72      }
73      err = pthread_cond_init(&barrier->cond, NULL);
74      if (err != 0) {
```

```
75          pthread_mutex_destroy(&barrier->mutex);
76          return err;
77      }
78      barrier->count = count;
79      barrier->waiting = 0;
80      barrier->generation = 0;
81      barrier->broken = 0;
82      return 0;
83  }
84
85  static void barrier_destroy(barrier_t* barrier) {
86      pthread_cond_destroy(&barrier->cond);
87      pthread_mutex_destroy(&barrier->mutex);
88  }
89
90  static int barrier_wait(barrier_t* barrier) {
91      pthread_mutex_lock(&barrier->mutex);
92      unsigned long generation = barrier->generation;
93      if (!barrier->broken) {
94          barrier->waiting++;
95          if (barrier->waiting == barrier->count) {
96              barrier->waiting = 0;
97              barrier->generation++;
98              pthread_cond_broadcast(&barrier->cond);
99          } else {
100             // Цикл защищает от ложных пробуждений pthread_cond_wait.
101             while (generation == barrier->generation && !barrier->broken) {
102                 pthread_cond_wait(&barrier->cond, &barrier->mutex);
103             }
104         }
105     }
106     int passed = generation != barrier->generation;
107     pthread_mutex_unlock(&barrier->mutex);
108     return passed ? 0 : -1;
109 }
110
111 static void barrier_break(barrier_t* barrier) {
112     pthread_mutex_lock(&barrier->mutex);
113     barrier->broken = 1;
114     pthread_cond_broadcast(&barrier->cond);
115     pthread_mutex_unlock(&barrier->mutex);
116 }
117
118 static double window_min(const double* input, size_t cols, const window_t* window) {
119     double result = input[(window->top * cols) + window->left];
120     for (size_t i = window->top; i <= window->bottom; i++) {
121         const double* row = input + (i * cols);
122         for (size_t j = window->left; j <= window->right; j++) {
123             if (row[j] < result) {
124                 result = row[j];
125             }
126         }
127     }
128     return result;
129 }
130
131 static double window_max(const double* input, size_t cols, const window_t* window) {
132     double result = input[(window->top * cols) + window->left];
```

```
133     for (size_t i = window->top; i <= window->bottom; i++) {
134         const double* row = input + (i * cols);
135         for (size_t j = window->left; j <= window->right; j++) {
136             if (row[j] > result) {
137                 result = row[j];
138             }
139         }
140     }
141     return result;
142 }
143 
144 // окно у края матрицы обрезается: учитываются только существующие элементы
145 static void filter_rows(
146     const morph_task_t* task,
147     morph_op_t operation,
148     const double* input,
149     double* output,
150     row_range_t rows
151 ) {
152     size_t cols = task->cols;
153     size_t radius = task->radius;
154     for (size_t i = rows.begin; i < rows.end; i++) {
155         window_t window = {
156             .top = i >= radius ? i - radius : 0,
157             .bottom = i + radius < task->rows ? i + radius : task->rows - 1,
158         };
159         for (size_t j = 0; j < cols; j++) {
160             window.left = j >= radius ? j - radius : 0;
161             window.right = j + radius < cols ? j + radius : cols - 1;
162             if (operation == kErode) {
163                 output[(i * cols) + j] = window_min(input, cols, &window);
164             } else {
165                 output[(i * cols) + j] = window_max(input, cols, &window);
166             }
167         }
168     }
169 }
170 
171 static void filter_both(const morph_task_t* task, size_t iteration, row_range_t rows)
        {
172     const double* eroded_in = iteration == 0 ? task->src : task->eroded[(iteration -
            1) % 2];
173     const double* dilated_in = iteration == 0 ? task->src : task->dilated[(iteration -
             1) % 2];
174     filter_rows(task, kErode, eroded_in, task->eroded[iteration % 2], rows);
175     filter_rows(task, kDilate, dilated_in, task->dilated[iteration % 2], rows);
176 }
177 
178 static void run_iteration(const worker_t* worker, size_t iteration) {
179     morph_task_t* task = worker->task;
180     if (!task->dynamic) {
181         filter_both(task, iteration, worker->band);
182         return;
183     }
184 
185     // Счётчик следующей итерации уже никто не трогает: все прошли барьер после
            предыдущей
186     // Обнуление станет видно остальным через следующий барьер
```

```
187     if (worker->index == 0) {
188         atomic_store_explicit(&task->next_chunk[(iteration + 1) % 2], 0,
                memory_order_relaxed);
189     }
190     for (;;) {
191         size_t chunk = atomic_fetch_add_explicit(&task->next_chunk[iteration % 2], 1,
                memory_order_relaxed);
192         if (chunk >= task->chunks) {
193             break;
194         }
195         row_range_t rows = {.begin = chunk * task->chunk_rows};
196         rows.end = rows.begin + task->chunk_rows < task->rows ? rows.begin + task->
                chunk_rows : task->rows;
197         filter_both(task, iteration, rows);
198     }
199 }
200
201 static void* worker_main(void* arg) {
202     const worker_t* worker = arg;
203     morph_task_t* task = worker->task;
204
205     if (barrier_wait(&task->barrier) != 0) {
206         return NULL;
207     }
208
209     for (size_t i = 0; i < task->iterations; i++) {
210         run_iteration(worker, i);
211         if (i + 1 < task->iterations && barrier_wait(&task->barrier) != 0) {
212             return NULL;
213         }
214     }
215     return NULL;
216 }
217
218 static int run_workers(morph_task_t* task, size_t threads) {
219     worker_t* workers = calloc(threads, sizeof(*workers));
220     if (workers == NULL) {
221         fprintf(stderr, "Not enough memory for %zu thread descriptors.\n", threads);
222         return -1;
223     }
224     int err = barrier_init(&task->barrier, threads);
225     if (err != 0) {
226         fprintf(stderr, "Cannot initialize the barrier: %s\n", strerror(err));
227         free(workers);
228         return -1;
229     }
230
231     for (size_t i = 0; i < threads; i++) {
232         workers[i].task = task;
233         workers[i].index = i;
234         workers[i].band.begin = i * task->rows / threads;
235         workers[i].band.end = (i + 1) * task->rows / threads;
236     }
237
238     size_t started = 1;
239     for (; started < threads; started++) {
240         err = pthread_create(&workers[started].thread, NULL, worker_main, &workers[
                started]);
```

```
        if (err != 0) {
            fprintf(stderr, "Cannot create thread %zu of %zu: %s\n", started + 1,
                threads, strerror(err));
            barrier_break(&task->barrier);
            break;
        }
    }

    worker_main(&workers[0]);

    int result = err == 0 ? 0 : -1;
    for (size_t i = 1; i < started; i++) {
        err = pthread_join(workers[i].thread, NULL);
        if (err != 0) {
            fprintf(stderr, "Cannot join thread %zu: %s\n", i + 1, strerror(err));
            result = -1;
        }
    }

    barrier_destroy(&task->barrier);
    free(workers);
    return result;
}

int morph_apply(
    const matrix_t* src,
    matrix_t* eroded,
    matrix_t* dilated,
    const morph_params_t* params,
    size_t* threads_used
) {
    size_t total = src->rows * src->cols;
    *threads_used = 0;
    if (params->iterations == 0) {
        memcpy(eroded->data, src->data, total * sizeof(double));
        memcpy(dilated->data, src->data, total * sizeof(double));
        return 0;
    }

    size_t threads = params->max_threads < src->rows ? params->max_threads : src->rows
        ;

    morph_task_t task = {
        .rows = src->rows,
        .cols = src->cols,
        .radius = params->window / 2,
        .iterations = params->iterations,
        .dynamic = !params->static_split,
        .src = src->data,
    };
    size_t target_chunks = threads * kChunksPerThread;
    task.chunk_rows = (src->rows + target_chunks - 1) / target_chunks;
    task.chunks = (src->rows + task.chunk_rows - 1) / task.chunk_rows;
    atomic_init(&task.next_chunk[0], 0);
    atomic_init(&task.next_chunk[1], 0);
    double* eroded_tmp = NULL;
    double* dilated_tmp = NULL;
    if (params->iterations > 1) {
```

```
        eroded_tmp = malloc(total * sizeof(double));
        dilated_tmp = malloc(total * sizeof(double));
        if (eroded_tmp == NULL || dilated_tmp == NULL) {
            fprintf(stderr, "Not enough memory for intermediate matrices.\n");
            free(eroded_tmp);
            free(dilated_tmp);
            return -1;
        }
    }
    size_t last = (params->iterations - 1) % 2;
    task.eroded[last] = eroded->data;
    task.eroded[1 - last] = eroded_tmp;
    task.dilated[last] = dilated->data;
    task.dilated[1 - last] = dilated_tmp;

    int result = run_workers(&task, threads);
    if (result == 0) {
        *threads_used = threads;
    }
    free(eroded_tmp);
    free(dilated_tmp);
    return result;
}
```

A.4. matrix.h

```
#ifndef MATRIX_H
#define MATRIX_H

#include <stddef.h>
#include <stdint.h>
#include <stdio.h>

// элемент (r, c) лежит в data[r * cols + c]
typedef struct {
    size_t rows;
    size_t cols;
    double* data;
} matrix_t;

int matrix_alloc(matrix_t* matrix, size_t rows, size_t cols);

void matrix_free(matrix_t* matrix);

// Формат: "rows cols", затем rows * cols чисел через пробелы
int matrix_read(FILE* input, matrix_t* matrix);

int matrix_write(FILE* out, const matrix_t* matrix);

void matrix_fill_random(matrix_t* matrix, uint64_t seed);

#endif
```

A.5. matrix.c

```
#include "matrix.h"

```

```c
#include <errno.h>
#include <math.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

enum {
    kTokenSize = 64,
};

// ограничение на одну сторону матрицы
static const size_t kMaxDimension = 1U << 20U;

int matrix_alloc(matrix_t* matrix, size_t rows, size_t cols) {
    matrix->rows = 0;
    matrix->cols = 0;
    matrix->data = NULL;
    if (rows == 0 || cols == 0 || rows > kMaxDimension || cols > kMaxDimension) {
        fprintf(stderr, "Invalid matrix size %zux%zu (each side must be 1..%zu).\n",
            rows, cols, kMaxDimension);
        return -1;
    }
    if (rows > SIZE_MAX / sizeof(double) / cols) {
        fprintf(stderr, "Matrix %zux%zu does not fit into the address space.\n", rows,
            cols);
        return -1;
    }

    matrix->data = malloc(rows * cols * sizeof(double));
    if (matrix->data == NULL) {
        fprintf(stderr, "Not enough memory for a %zux%zu matrix.\n", rows, cols);
        return -1;
    }
    matrix->rows = rows;
    matrix->cols = cols;
    return 0;
}

void matrix_free(matrix_t* matrix) {
    free(matrix->data);
    matrix->data = NULL;
    matrix->rows = 0;
    matrix->cols = 0;
}

static int read_token(FILE* input, char token[kTokenSize]) {
    int scanned = fscanf(input, "%63s", token);
    if (scanned == 1) {
        return 1;
    }
    if (ferror(input)) {
        perror("Failed to read the matrix");
        return -1;
    }
    return 0;
}

static int read_dimension(FILE* input, const char* name, size_t* value) {
```

```
    char token[kTokenSize];
    int status = read_token(input, token);
    if (status <= 0) {
        if (status == 0) {
            fprintf(stderr, "Matrix header: missing %s.\n", name);
        }
        return -1;
    }

    char* end = NULL;
    errno = 0;
    unsigned long long parsed = strtoull(token, &end, 10);
    if (token[0] == '-' || token[0] == '+' || end == token || *end != '\0' || errno ==
         ERANGE || parsed > SIZE_MAX) {
        fprintf(stderr, "Matrix header: invalid %s '%s'.\n", name, token);
        return -1;
    }
    *value = (size_t)parsed;
    return 0;
}

int matrix_read(FILE* input, matrix_t* matrix) {
    size_t rows = 0;
    size_t cols = 0;
    if (read_dimension(input, "row count", &rows) != 0 || read_dimension(input, "
        column count", &cols) != 0) {
        return -1;
    }
    if (matrix_alloc(matrix, rows, cols) != 0) {
        return -1;
    }

    size_t total = rows * cols;
    char token[kTokenSize];
    for (size_t i = 0; i < total; i++) {
        int status = read_token(input, token);
        if (status <= 0) {
            if (status == 0) {
                fprintf(stderr, "Matrix data: expected %zu numbers, got %zu.\n", total,
                     i);
            }
            matrix_free(matrix);
            return -1;
        }

        char* end = NULL;
        double value = strtod(token, &end);
        if (end == token || *end != '\0' || isnan(value)) {
            fprintf(stderr, "Matrix data: invalid number '%s' at row %zu, column %zu.\n
                ", token, i / cols, i % cols);
            matrix_free(matrix);
            return -1;
        }
        matrix->data[i] = value;
    }

    int status = read_token(input, token);
    if (status != 0) {
```

```c
        if (status == 1) {
            fprintf(stderr, "Matrix data: unexpected extra value '%s'.\n", token);
        }
        matrix_free(matrix);
        return -1;
    }
    return 0;
}

int matrix_write(FILE* out, const matrix_t* matrix) {
    if (fprintf(out, "%zu %zu\n", matrix->rows, matrix->cols) < 0) {
        return -1;
    }
    for (size_t i = 0; i < matrix->rows; i++) {
        const double* row = matrix->data + (i * matrix->cols);
        for (size_t j = 0; j < matrix->cols; j++) {
            // 15 значащих цифр восстанавливают любое десятичное число, введённое с не
                большей точностью.
            if (fprintf(out, j + 1 < matrix->cols ? "%.15g " : "%.15g\n", row[j]) < 0)
                {
                return -1;
            }
        }
    }
    return fflush(out) == 0 ? 0 : -1;
}

// splitmix64 - воспроизводимый по сиду
static uint64_t next_random(uint64_t* state) {
    uint64_t mixed = (*state += 0x9E3779B97F4A7C15ULL);
    mixed = (mixed ^ (mixed >> 30U)) * 0xBF58476D1CE4E5B9ULL;
    mixed = (mixed ^ (mixed >> 27U)) * 0x94D049BB133111EBULL;
    return mixed ^ (mixed >> 31U);
}

void matrix_fill_random(matrix_t* matrix, uint64_t seed) {
    uint64_t state = seed;
    size_t total = matrix->rows * matrix->cols;
    for (size_t i = 0; i < total; i++) {
        matrix->data[i] = (double)(next_random(&state) % 10000U) / 100.0;
    }
}
```

A.6. Makefile

```
CC ?= cc
CFLAGS ?= -std=c11 -Wall -Wextra -Wpedantic -O2
CPPFLAGS += -D_POSIX_C_SOURCE=200809L -Isrc -pthread
LDFLAGS += -pthread
LDLIBS += -lm

vpath %.c src
vpath %.h src

all: morph

morph: main.o matrix.o morph.o
  $(CC) $(LDFLAGS) -o $@ $^ $(LDLIBS)
```

```
14 |
15 | main.o: matrix.h morph.h
16 | matrix.o: matrix.h
17 | morph.o: matrix.h morph.h
18 |
19 | clean:
20 |   rm -f morph *.o
21 |
22 | .PHONY: all clean
```